

TechTribers

Java Interview Series · Part 21 · Q1035–Q1086

Git Interview Questions

From the object model to production incidents: the three areas, branching, merge versus rebase, undoing and recovering work, force-push safety, workflows, hooks, signing, large repositories, CI pipelines, and Git in Java and Spring Boot projects.

52	52	5	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `GT-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

52	6	52	5
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🧬	01	Git Fundamentals and Internals	Q1035–Q1044	5
🔀	02	Branching, Merging and Rebasing	Q1045–Q1054	13
🛟	03	Undoing Things and Recovering Work	Q1055–Q1063	21
🌍	04	Remotes, Tags and Workflows	Q1064–Q1073	28
🛡️	05	Hooks, Security and Scale	Q1074–Q1079	35
🏭	06	Git in Teams, CI/CD and Java Projects	Q1080–Q1086	40

© TechTribers. For personal use only. Do not redistribute.

All questions

🧬 Git Fundamentals and Internals

Q1035 What problem does Git solve, and how does a distributed VCS differ from a centralized one like SVN? 5

Q1036 What are the working tree, the index (staging area) and the repository? 6

Q1037 How does Git store data? Explain blobs, trees, commits and tags. 7

Q1038 What are HEAD, branches and a detached HEAD, and how do you recover work done in one? 8

Q1039 What do `git switch` and `git restore` do, and why were they added to replace most of `git checkout`? 8

Q1040 What is the difference between `git fetch`, `git pull` and `git pull --rebase`? 9

Q1041 How does .gitignore work, and how do you stop tracking a file that's already committed? 10

Q1042 What do the two-dot and three-dot forms mean in `git diff` and `git log`? 10

Q1043 How do you search history: log filters, pickaxe, line history and blame? 11

Q1044 Which Git configuration settings should every developer set? 12

🔀 Branching, Merging and Rebasing

Q1045 Merge vs rebase: what is the difference, and which should you use? 13

Q1046 How does interactive rebase work, and how do fixup commits and autosquash make it easy? 14

Q1047 How do you resolve merge conflicts, and what are rerere and zdiff3? 15

Q1048 What does `git rebase --onto` do, and how do you manage stacked branches? 16

Q1049 Squash merge vs merge commit vs rebase merge for pull requests: how do you choose? 16

Q1050 How does `git cherry-pick` work, and how do you use it for backports? 17

Q1051 How do you revert a merge commit, and why does re-merging the branch afterwards bring nothing back? 18

Q1052 How does `git stash` work, and how do you use it safely? 18

Q1053 What is `git worktree`, and when is it better than stashing or cloning again? 19

Q1054 How should you keep a feature branch up to date with main: merge or rebase? 20

🛟 Undoing Things and Recovering Work

Q1055 reset vs revert vs restore vs clean: which undo do you use when? 21

Q1056 How do you amend the last commit, fix an older one, or change author and message? 22

Q1057 How does the reflog work, and how do you recover commits after `reset --hard` or a bad rebase? 23

Q1058 You committed to the wrong branch (for example main instead of a feature branch). How do you fix it? 23

Q1059 How do you recover a deleted branch, a deleted file or a dropped stash? 24

Q1060 How does `git bisect` find the commit that introduced a bug, and how do you automate it? 25

Q1061 How do you find who changed something, when, and why? 26

Q1062 How do you remove a secret or a large file from Git history? 26

Q1063 What do `git fsck`, `git gc` and `git prune` do, and what if the repository is corrupted? 27

🌍 Remotes, Tags and Workflows

Q1064 What are remotes, remote-tracking branches and upstream branches? 28

Q1065 When is force-pushing acceptable, and what do `--force-with-lease` and `--force-if-includes` add? 29

Q1066 What is the difference between lightweight and annotated tags, and how do you use tags for releases? 29

Q1067 Gitflow vs GitHub flow vs trunk-based development: which should a team use? 30

Q1068 How does the fork-and-pull-request workflow work, and how do you keep a fork in sync? 31

Q1069 What makes a good commit and commit message, and what are Conventional Commits? 32

Q1070 Submodules vs subtrees vs publishing a library: how do you share code between repositories? 32

Q1071 How do line endings and `.gitattributes` work, and how do you avoid CRLF/LF diffs? 33

Q1072 What is Git LFS, how does it work, and what are its pitfalls? 34

Q1073 Your local branch and the remote branch have diverged. What do you do? 34

© TechTribers. For personal use only. Do not redistribute.

🛡️ Hooks, Security and Scale

Q1074 What are Git hooks, and how do you share them across a team? 35

Q1075 Why and how do you sign commits (GPG, SSH, gitsign), and how do you verify them? 36

Q1076 How are objects stored in packfiles, and why do repositories get huge? 37

Q1077 How do shallow, partial and sparse clones help with large repositories and monorepos? 37

Q1078 Does Git's use of SHA-1 make it insecure, and what is SHA-256 mode? 38

Q1079 SSH keys vs HTTPS tokens vs credential helpers: how do you authenticate, including multiple accounts? 39

🏭 Git in Teams, CI/CD and Java Projects

Q1080 What do branch protection, rulesets and CODEOWNERS enforce? 40

Q1081 How do you use Git correctly in CI pipelines (fetch depth, changed files, versions)? 41

Q1082 How do release branches, hotfixes and backports work in practice? 41

Q1083 How do you embed Git commit information in a Spring Boot build (git.properties, /actuator/info)? 42

Q1084 What should a .gitignore and the committed files look like for a Java/Maven/Gradle project? 43

Q1085 How do you version a Maven project from Git tags (release plugin vs tag-driven CI)? 44

Q1086 Interview scenario: you accidentally pushed a password to `main`. What do you do, in order? 45

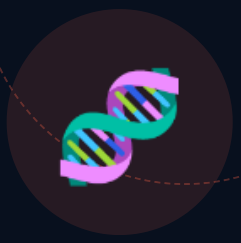
© TechTribers. For personal use only. Do not redistribute.

Section 01

Git Fundamentals and Internals

Distributed versus centralized version control, the three areas, the object model (blob, tree, commit, tag), HEAD and detached HEAD, switch and restore, fetch versus pull, ignoring files, the dot-versus-three-dot diff rules, searching history and the configuration every developer should set.

Questions 1035–1044 · 10 questions



Q1035 GT-001 • Basic

What problem does Git solve, and how does a distributed VCS differ from a centralized one like SVN?

⚡ Quick answer

Git tracks the history of a project so many people can change it in parallel and merge the results. In a **centralized** VCS (SVN, CVS) the server holds the history and your machine holds a working copy; most operations need the server. In **Git every clone is a complete repository** with the full history, so commits, diffs, logs and branching are **local and fast**, work offline, and every clone is a backup. Branches are cheap pointers, not directory copies, and integrity is enforced by hashing every object.

The distributed model separates **recording** (commit, local) from **sharing** (push/pull). That's why Git workflows are built around branches and pull requests, and why a mistake in a local commit can be fixed freely until you push.

Terminal

```
git clone https://github.com/acme/orders.git     # full history, not just the latest files
cd orders
git log --oneline | head                          # offline: history is local
git switch -c feature/refunds                     # a branch is a 41-byte file: instant
git commit -am "Add refund endpoint"              # local commit, nothing sent anywhere
git push -u origin feature/refunds                # sharing is an explicit, separate step
```


© TechTribers. For personal use only. Do not redistribute.

Q1036 GT-002 • Basic

What are the working tree, the index (staging area) and the repository?

⚡ **Quick answer**

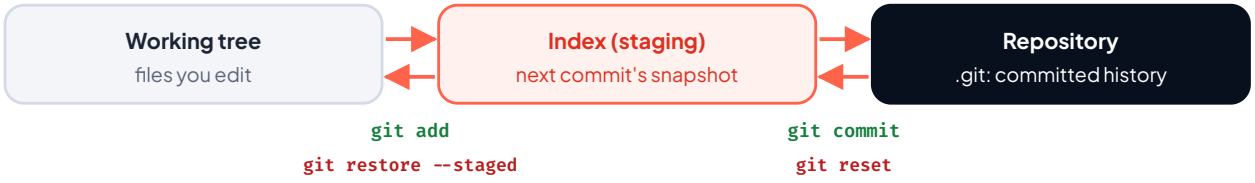
Git has three areas. The **working tree** holds the files you edit. The **index (staging area)** holds the snapshot you're **preparing for the next commit**. The **repository** (`.git`) stores committed history. `git add` copies changes from the working tree to the index, `git commit` turns the index into a commit, and `git restore` moves content the other way. The index lets you commit **part** of your changes.

Terminal

```
git status -sb                    # short status with branch info
git add src/Order.java            # working tree -> index
git add -p                        # stage selected hunks only
git diff                          # working tree vs index (unstaged changes)
git diff --staged                 # index vs HEAD (what the next commit will contain)
git commit -m "Validate refund amount"    # index -> repository

git restore --staged src/Order.java       # index <- HEAD (unstage, keep file changes)
git restore src/Order.java                # working tree <- index (discard edits!)
```

Where changes live, and which command moves them



`git diff = working tree vs index · git diff --staged = index vs HEAD`
`git restore <file> = index → working tree (discards edits) · git add -p = stage part of a file`

💡 **Pro tip**

Review with `git diff --staged` before every commit. It's the cheapest way to catch debug prints, accidental files and half-finished hunks.

© TechTribers. For personal use only. Do not redistribute.

Q1037 GT-003

• Advanced

How does Git store data? Explain blobs, trees, commits and tags.

⚡ Quick answer

Git is a **content-addressable store**: every object is saved under the SHA hash of its content. A **blob** is a file's content (no name). A **tree** maps names to blobs and sub-trees (a directory snapshot). A **commit** points to one **tree** (the full project snapshot), its **parent commit(s)**, author, committer and message. An **annotated tag** points to a commit with a message. Branches and tags are just **refs**: files containing a commit hash.

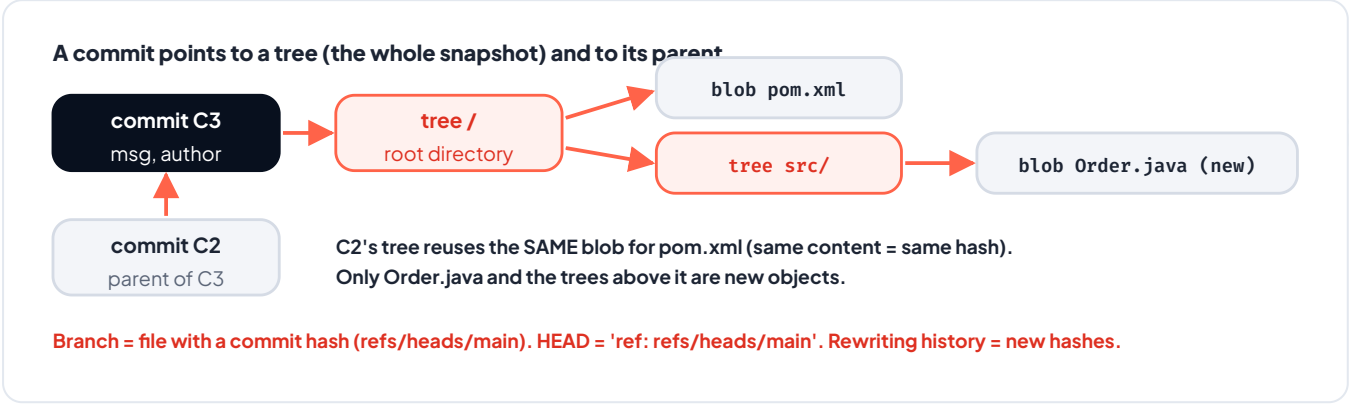
Consequences worth stating in an interview:

- **Snapshots, not diffs.** Each commit references a complete tree; unchanged files reuse the same blob, so nothing is duplicated. Packfiles later store similar objects as deltas purely as a storage optimisation (Q1076).
- **Identical content = identical hash**, so renaming a file doesn't create new content; Git **detects renames at diff time** by similarity, it doesn't record them.
- **Changing anything changes the hash**, including parents, so rewriting history (amend, rebase) creates new commits and the old ones become unreachable, not edited.

Terminal

```
git cat-file -p HEAD                # the commit: tree, parent, author, message
git cat-file -p HEAD^{tree}          # the root tree: modes, types, hashes, names
git cat-file -t 5a1c3f9              # object type: blob | tree | commit | tag
git ls-tree -r HEAD --name-only      # every file in the snapshot

echo "hello" | git hash-object --stdin      # the id a blob with this content would get
cat .git/refs/heads/main                    # a branch: just a commit hash
cat .git/HEAD                               # ref: refs/heads/main
```



Branch = file with a commit hash (refs/heads/main). HEAD = 'ref: refs/heads/main'. Rewriting history = new hashes.

© TechTribers. For personal use only. Do not redistribute.

📍 Q1038 GT-004

• Intermediate

What are HEAD, branches and a detached HEAD, and how do you recover work done in one?

⚡ **Quick answer**

A **branch** is a movable pointer to a commit. **HEAD** is a pointer to what's checked out, normally a **symbolic ref to the current branch**; committing moves that branch forward. In a **detached HEAD** (after checking out a commit, tag or `origin/main` directly) HEAD points to a commit with no branch, so new commits belong to nothing and become unreachable as soon as you switch away. Fix: create a branch before leaving.

Terminal

```
git switch --detach v2.3.0       # detached HEAD: inspect or build an old release
# ... experiment, commit something useful ...
git switch -c hotfix/2.3.1         # keep it: a branch now points at these commits

# Already switched away and "lost" the commits?
git reflog                          # find the commit (Q1057)
git branch rescue 7d3f2a1           # point a branch at it again
```

⚠️ **Interview trap**

Committing in a detached HEAD, switching to `main`, and finding the work gone. It isn't deleted (the reflog keeps it for about 30 days), but nothing references it. Git warns when you leave; read the warning.

🆕 Q1039 GT-005

• Basic

What do `git switch` and `git restore` do, and why were they added to replace most of `git checkout`?

⚡ **Quick answer**

`git checkout` did several unrelated things: switch branches, detach HEAD, and overwrite files from the index or a commit, which made it easy to destroy work by accident. Git 2.23 split it: `git switch` changes branches, `git restore` restores files. `checkout` still works, but the new commands are clearer and safer.

Terminal

```
git switch main                    # change branch
git switch -c feature/audit         # create and switch
git switch -                        # back to the previous branch
git switch --detach v2.3.0          # inspect a tag

git restore src/Order.java          # discard working-tree changes (destructive)
git restore --staged src/Order.java # unstage
```


© TechTribers. For personal use only. Do not redistribute.

```
git restore --source=HEAD~2 pom.xml    # file as it was two commits ago, into the working tree
git restore -p .                       # discard selected hunks interactively
```

> 💡 **Pro tip**
> `git restore <file>` has no undo for uncommitted edits. Run `git diff <file>` first, or `git stash` if you're not sure.

⬇️ Q1040 GT-006 • Basic

What is the difference between `git fetch`, `git pull` and `git pull --rebase`?

> ⚡ **Quick answer**
> `git fetch` downloads new commits and updates **remote-tracking branches** (`origin/main`) without touching your branches or files. `git pull` is **fetch + merge** into the current branch (creating merge commits when histories diverge). `git pull --rebase` is **fetch + rebase**: your local commits are replayed on top of the fetched ones, keeping history linear. `git pull --ff-only` refuses anything that isn't a fast-forward, which is the safest default.

Terminal

```
git fetch origin                        # safe: only updates origin/* refs
git log --oneline main..origin/main      # what would I get?
git diff main...origin/main              # and what changed?

git pull                                 # fetch + merge
git pull --rebase                        # fetch + rebase my local commits
git pull --ff-only                       # fail instead of creating a merge commit

# Make rebase or ff-only the default
git config --global pull.rebase true
git config --global pull.ff only          # alternative: error on divergence
```

> ⚠️ **Interview trap**
> A plain `git pull` on a branch with local commits quietly creates a merge commit titled "Merge branch 'main' of ...". Teams that haven't set `pull.rebase` or `pull.ff` end up with those noise commits all over history.

© TechTribers. For personal use only. Do not redistribute.

Q1041 GT-007

• Basic

How does .gitignore work, and how do you stop tracking a file that's already committed?

⚡ Quick answer

`.gitignore` lists patterns of **untracked** files Git should not offer to add. It has **no effect on files already tracked**: remove them from the index with `git rm --cached` (the file stays on disk), commit, and then the ignore rule applies. Use the global ignore file (`core.excludesFile`) for personal clutter such as `.DS_Store` and editor files, and `.git/info/exclude` for local-only rules.

Terminal

```
# .gitignore
target/
*.log
.env
!config/.env.example        # a negation: keep this one
/build/                      # only at the repository root

git rm --cached .env                 # stop tracking, keep the local file
git rm -r --cached .idea/            # untrack a directory
git commit -m "Stop tracking local config"

git check-ignore -v target/app.jar   # which rule ignores this path?
git config --global core.excludesFile ~/.gitignore_global
git ls-files --others --ignored --exclude-standard    # list ignored files
```

⚠️ Interview trap

Adding a secret file to `.gitignore` after it was committed. It's still in history and in every clone. Rotate the secret and purge it properly (Q1062).

Q1042 GT-008

• Advanced

What do the two-dot and three-dot forms mean in `git diff` and `git log`?

⚡ Quick answer

They mean different things in the two commands. For `git log A..B` : commits reachable from B but not from A. For `git log A...B` : commits in either but not both (symmetric difference). For `git diff A..B` : the same as `git diff A B` (difference between the two tips). For `git diff A...B` : the changes on B **since the merge base** with A, which is exactly what a pull request shows.

Terminal

```
git log --oneline main..feature      # commits the PR would add
git log --oneline feature..main      # commits the PR branch is missing
```


© TechTribers. For personal use only. Do not redistribute.

```
git log --oneline --left-right main...feature    # both sides, marked < and >

git diff main..feature               # tip vs tip (includes unrelated changes on main)
git diff main...feature              # only what feature changed since it branched
git diff --stat main...feature       # review size of the PR

git merge-base main feature          # the common ancestor used by three-dot
```

> 💡 **Pro tip**
> To review a branch the way the code-hosting site does, use `git diff origin/main...HEAD`. The two-dot form also shows the reverse of everything that landed on main after you branched.

🔎 Q1043 GT-009 • Intermediate

How do you search history: log filters, pickaxe, line history and blame?

> ⚡ **Quick answer**
> `git log` can filter by author, date, message and path, and shape its output. `-S'text'` (pickaxe) finds commits that changed the **number of occurrences** of a string; `-G'regex'` finds commits whose diff matches a regex; `-L :method:File.java` shows the history of one function or line range; `--follow` tracks a file across renames. `git blame` shows who last changed each line, with `-w` ignoring whitespace and `-C` detecting moved lines.

Add a `.git-blame-ignore-revs` file listing bulk formatting commits and set `git config blame.ignoreRevsFile .git-blame-ignore-revs`; blame then skips them (GitHub honours that file too).

Terminal

```
git log --oneline --graph --decorate --all        # the topology at a glance
git log --author="asha" --since="2 weeks ago" --no-merges --oneline
git log --grep="JIRA-1234" --oneline                  # search commit messages
git log -S"retryCount" --oneline -- src/              # who added or removed this symbol?
git log -G"timeout\s*=\s*\d+" -p -- application.yml   # diffs matching a regex
git log -L :calculateTotal:src/main/java/Order.java   # history of one method
git log --follow -p -- src/main/java/Invoice.java     # across renames

git blame -L 40,70 -w -C src/main/java/Order.java     # who wrote lines 40-70?
git log --first-parent --oneline main                 # only merge/PR-level history
```


© TechTribers. For personal use only. Do not redistribute.

Q1044 GT-010

• Intermediate

Which Git configuration settings should every developer set?

⚡ **Quick answer**

Config has levels: `--system`, `--global` (your user), `--local` (this repo, the default) and `--worktree`; more specific levels override. Beyond name and email, the settings that prevent real problems are a default branch name, `pull.rebase` / `pull.ff`, `push.autoSetupRemote`, `fetch.prune`, `rerere.enabled`, a three-way conflict style and autosquash for fixups.

Terminal

```
git config --global user.name  "Asha Kulkarni"
git config --global user.email "asha@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase true               # or: pull.ff only
git config --global push.autoSetupRemote true      # Git 2.37+: first push sets upstream
git config --global fetch.prune true               # drop remote-tracking refs of deleted branches
git config --global rerere.enabled true            # remember conflict resolutions (Q1047)
git config --global merge.conflictStyle zdiff3     # Git 2.35+: show the base in conflicts
git config --global rebase.autoSquash true         # use fixup! commits automatically
git config --global rebase.updateRefs true         # Git 2.38+: move stacked branches too
git config --global diff.algorithm histogram       # often cleaner diffs

# Different identity for work repositories
git config --global includeIf.gitdir:~/work/.path ~/.gitconfig-work

git config --list --show-origin                    # where does each value come from?
git config --global alias.lg "log --oneline --graph --decorate --all"
```

⚠️ **Interview trap**

Committing with the wrong email (a personal address in a company repo, or no GitHub-linked email). History rewriting is the only fix afterwards; use `includeIf` so the right identity is automatic.

© TechTribers. For personal use only. Do not redistribute.

Section 02

Branching, Merging and Rebasing

Merge versus rebase, interactive rebase and fixup, resolving conflicts with rerere, rebase --onto for stacked branches, squash versus merge commit versus rebase merge, cherry-pick backports, reverting a merge, stash, worktrees and keeping a feature branch current.

Questions 1045–1054 · 10 questions

Q1045 GT-011

• Intermediate

Merge vs rebase: what is the difference, and which should you use?

⚡ Quick answer

`git merge` joins two histories: if the target hasn't moved it just **fast-forwards**, otherwise it creates a **merge commit** with two parents, preserving exactly what happened. `git rebase` **replays** your commits one by one on top of another base, producing **new commits** (new hashes) and a **linear** history. Rebase for tidying **your own unpublished** branch; merge (or a hosting-site merge strategy) to integrate shared work. Rule: **never rebase commits other people already have**.

Merge keeps original commit hashes (safe for shared branches) and records when integration happened, but busy repositories fill up with merge commits. Rebase gives a clean `git log` and simpler `bisect`, but rewrites history, so after rebasing a branch you've already pushed you must force-push it (Q1065).

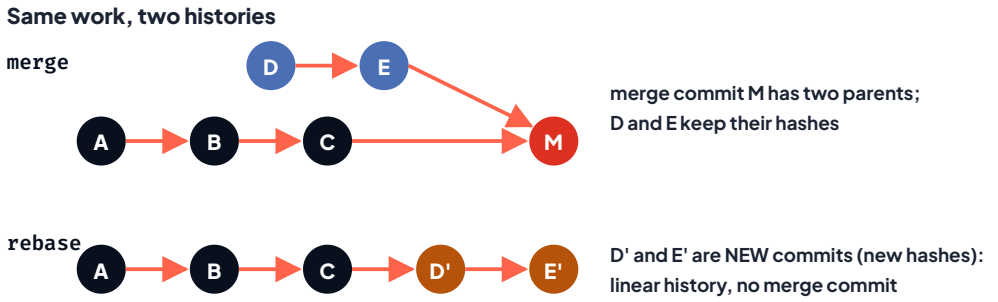
A common team convention: **rebase your own feature branch** onto main before the PR, then merge through the PR with **squash or rebase** (Q1049).

Terminal

```
# Merge: keep the true shape of history
git switch main
git merge --no-ff feature/refunds        # always create a merge commit
git merge --ff-only feature/refunds      # only if a fast-forward is possible

# Rebase: replay my commits on the latest main, then fast-forward main
git switch feature/refunds
git rebase main                          # my commits get new hashes
git switch main
git merge --ff-only feature/refunds      # linear history, no merge commit
```


© TechTribers. For personal use only. Do not redistribute.



Rule: never rebase commits other people already have. Rebased pushed branch → `--force-with-lease`.

⚠️ **Interview trap**

Rebasing a shared branch (`main` , `develop` , a branch two people work on). Everyone else's copy now contains the old commits and Git sees two parallel histories: duplicate commits and painful merges.

Q1046 GT-012 • Intermediate

How does interactive rebase work, and how do fixup commits and autosquash make it easy?

⚡ **Quick answer**

`git rebase -i HEAD~5` opens a todo list of the last five commits that you can **reorder**, `reword` , `edit` , `squash` , `fixup` **or** `drop` before Git replays them. A `fixup!` commit created with `git commit --fixup <sha>` plus `git rebase -i --autosquash` (or `rebase.autoSquash=true`) moves each fixup next to its target and squashes it automatically. Only rewrite commits you haven't shared.

Terminal

```
git rebase -i origin/main              # edit everything since branching from main

# The todo list (oldest first):
#   pick   a1b2c3d Add Refund entity
#   fixup  e4f5a6b Fix typo in Refund entity
#   reword 7c8d9e0 refund stuff
#   drop   1f2e3d4 Debug logging
#   squash 5a6b7c8 Add refund validation

# Cheaper workflow: record fixes as you go, squash once at the end
git commit --fixup a1b2c3d                # creates "fixup! Add Refund entity"
git rebase -i --autosquash origin/main    # fixups are reordered and squashed for you

git rebase -i -x "mvn -q test" origin/main    # run tests after every commit while replaying
git rebase --abort                            # something went wrong: back to the start
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> After `rebase -i`, `git range-diff main@{1} main HEAD` (or `ORIG_HEAD`) compares the old and new series, so you can check that you changed what you meant to.

⚔️ Q1047 GT-013 • Intermediate

How do you resolve merge conflicts, and what are rerere and zdiff3?

> ⚡ **Quick answer**
> A conflict means both sides changed the same lines (or one deleted what the other edited). Git marks the file with `<<<<<<<`, `=======`, `>>>>>>>`. Edit to the final content, `git add` the file, then continue (`git merge --continue`, `git rebase --continue`). `rerere` (reuse recorded resolution) remembers how you resolved a conflict and **re-applies it** the next time the same conflict appears, which matters when rebasing a long branch repeatedly. `zdiff3` conflict style also shows the **common ancestor** version, which makes the right answer much clearer.

Terminal

```
git merge main
# CONFLICT (content): Merge conflict in src/main/java/Order.java
git status                          # "both modified" files
git diff --name-only --diff-filter=U    # list unresolved files

# <<<<<<< HEAD (yours: the branch you are on)
# ||||||| base                      (zdiff3 only: the original)
# =======
# >>>>>>> main (theirs)

git add src/main/java/Order.java
git merge --continue                # or: git rebase --continue / git cherry-pick --continue
git merge --abort                   # give up and return to the pre-merge state

git checkout --ours   pom.xml       # take our version of a file (see the warning below)
git mergetool                       # open the configured visual tool
git config --global rerere.enabled true
```

> ⚠️ **Interview trap**
> During a **rebase**, `ours` and `theirs` are **swapped**: `ours` is the branch you're rebasing **onto**, and `theirs` is **your** commit being replayed. Using `--theirs` while rebasing therefore keeps your side, the opposite of what it means in a merge.

© TechTribers. For personal use only. Do not redistribute.

Q1048 GT-014

• Advanced

What does `git rebase --onto` do, and how do you manage stacked branches?

⚡ Quick answer

`git rebase --onto <newbase> <oldbase> <branch>` takes the commits **after** `oldbase` **up to** `branch` and replays them on `newbase`. Use it to move a branch to a different parent: for example when `feature-b` was started from `feature-a`, and `feature-a` has since been squash-merged into `main`, plain `rebase main` would try to replay A's commits again and conflict. With **stacked** branches, `rebase.updateRefs` / `--update-refs` (Git 2.38) moves the intermediate branches along automatically.

The mental model: `--onto` is a three-argument rebase: *"take commits in (oldbase, branch], put them on newbase"*.

Terminal

```
# main --- A1 --- A2          (feature-a)
#                  \
#                   B1 --- B2   (feature-b, started from feature-a)

# feature-a was squash-merged into main as S. Move B1,B2 onto main:
git rebase --onto main feature-a feature-b

# Rebase a whole stack and move every branch in it
git switch feature-c
git rebase --update-refs main        # feature-a, feature-b and feature-c all move

# Drop a range of commits from the middle of a branch
git rebase --onto HEAD~5 HEAD~3      # removes the 2 commits between HEAD~5 and HEAD~3
```

Q1049 GT-015

• Intermediate

Squash merge vs merge commit vs rebase merge for pull requests: how do you choose?

⚡ Quick answer

Merge commit: keeps every branch commit and adds a merge commit; exact history, noisy. **Squash and merge**: collapses the PR into **one commit** on main with a new message; clean, one revertable unit, but loses the individual commits and authorship granularity, and the feature branch **diverges** from main afterwards. **Rebase and merge**: replays the PR's commits onto main without a merge commit: linear history that keeps the commits, but rewrites their hashes. Pick one policy per repository.

- **Squash**: best when PR commits are messy (`wip`, `fix`, `fix2`). Each main commit = one reviewed change, easy to revert and bisect. Write the squash message deliberately.
- **Rebase merge**: best when authors curate **atomic commits**; otherwise you publish the `wip` commits.
- **Merge commit**: best for long-lived branches or release branches where the branch's own history matters.

© TechTribers. For personal use only. Do not redistribute.

Enforce the choice with branch protection settings ("allow squash merging" only) rather than convention (Q1080).

Terminal

```
# What each hosting-site button does, in plain Git
# Merge commit
git switch main && git merge --no-ff feature/refunds

# Squash and merge
git switch main && git merge --squash feature/refunds
git commit -m "Add refund endpoint (#482)"

# Rebase and merge
git switch feature/refunds && git rebase main
git switch main && git merge --ff-only feature/refunds
```

⚠️ **Interview trap**

Continuing to work on a branch after it was squash-merged. Git doesn't know the squash commit contains your changes, so the next PR re-applies them and conflicts. Start new work from a fresh branch off main (or use `rebase --onto`, Q1048).

Q1050 GT-016 • Intermediate

How does `git cherry-pick` work, and how do you use it for backports?

⚡ **Quick answer**

`git cherry-pick <sha>` applies the **change introduced by a commit** onto your current branch as a **new commit** (different hash). Use it for **backports** and hotfixes: land the fix on `main`, then cherry-pick it onto `release/2.4`. `-x` appends the original hash to the message, `-n` applies without committing, ranges use `A^..B`, and a **merge commit** needs `-m 1` to say which parent is the mainline.

Terminal

```
git switch release/2.4
git cherry-pick -x 9f3a7c1                # message gains: (cherry picked from commit 9f3a7c1...)
git cherry-pick -x 4b2e8d0..a77c1f3       # a range (exclusive start); use A^..B to include A
git cherry-pick -n 9f3a7c1 5d6e7f8        # stage both changes, commit them together

git cherry-pick -m 1 c0ffee1              # a merge commit: diff against its first parent

git cherry-pick --continue                # after resolving conflicts
git cherry-pick --abort
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Cherry-picking a fix to a release branch **before** it is on `main`. The next release is cut from `main` without the fix and the bug returns. Policy: fix on main first (upstream-first), then backport.

↩️ Q1051 GT-017 • Advanced

How do you revert a merge commit, and why does re-merging the branch afterwards bring nothing back?

⚡ **Quick answer**
Reverting a merge needs `-m 1` to pick the mainline parent (the branch that should keep its state): `git revert -m 1 <merge-sha>`. The revert is a new commit that undoes the **changes the branch brought in**. But Git still records those commits as **already merged**, so merging the fixed branch again adds **only commits made after the original merge**, and the earlier work stays reverted. To bring it back, **revert the revert**, then merge.

The underlying rule: merge decides what to bring in by **commit ancestry, not by file content**. A reverted merge is still in the branch's ancestry.

Terminal

```
git revert -m 1 8e2f4a6           # undo the feature's changes on main
git push origin main                 # production is safe again

# Weeks later the branch is fixed. A plain merge would silently skip the old commits:
git switch main
git revert <sha-of-the-revert>       # "Revert 'Revert merge ...'" restores the original changes
git merge feature/refunds            # now only the new fixes need merging

# Alternative: recreate the branch with new commit hashes
git switch feature/refunds
git rebase -i main                   # rewritten commits are seen as new work
```

🧳 Q1052 GT-018 • Basic

How does `git stash` work, and how do you use it safely?

⚡ **Quick answer**
`git stash` saves **uncommitted changes** (tracked files by default) as a commit-like entry and resets the working tree to HEAD, so you can switch tasks. Restore with `apply` (keeps the stash) or `pop` (applies, then drops it if there's no conflict). Include untracked files with `-u`, name it with `-m`, stash selected hunks with `-p`, and turn a stash into a branch with `git stash branch`. Stashes are **local** and are not pushed.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
git stash push -m "WIP refund validation"      # tracked changes
git stash push -u -m "WIP with new files"        # include untracked files
git stash push -p                                 # choose hunks interactively
git stash push --keep-index                       # stash only what is NOT staged

git stash list                                    # stash@{0} is the newest
git stash show -p stash@{1}                       # the actual diff
git stash apply stash@{1}                         # restore, keep the stash
git stash pop                                     # restore and drop (stays if it conflicts)
git stash branch feature/refunds-wip stash@{0}    # new branch from the stash's base commit
git stash drop stash@{0}
```

⚠️ **Interview trap**

Relying on stashes as storage. They have no names beyond your message, they're easy to drop by mistake, and a conflict on `pop` keeps the stash but leaves a half-applied working tree. For anything that matters, commit to a temporary branch.

🌲 Q1053 GT-019

• Intermediate

What is `git worktree`, and when is it better than stashing or cloning again?

⚡ **Quick answer**

`git worktree add` checks out **another branch into a separate directory that shares the same repository** (objects, refs, config). You can review a pull request, run a long build or fix a hotfix **without stashing** and without a second full clone. Each branch can be checked out in only one worktree at a time.

Terminal

```
git worktree add ../orders-hotfix hotfix/2.4.1           # existing branch in a new directory
git worktree add -b feature/audit ../orders-audit origin/main  # create a branch too
git worktree list
cd ../orders-hotfix && mvn -q verify                      # build there while main keeps running

git worktree remove ../orders-hotfix                      # after merging
git worktree prune                                        # forget deleted directories
```

💡 **Pro tip**

Worktrees are also how AI coding agents and CI tools work on several branches in parallel safely: each task gets its own directory and cannot trample another's files.

© TechTribers. For personal use only. Do not redistribute.

🔄 **Q1054** GT-020 • Intermediate

How should you keep a feature branch up to date with main: merge or rebase?

> ⚡ **Quick answer**
> If the branch is **private** (only you, not yet reviewed), **rebase onto the latest main**: linear history and no merge commits. If the branch is **shared** or already in review with comments attached to commits, **merge main into it**: no rewriting, nothing to force-push. Whichever you choose, do it **regularly** so conflicts stay small, and decide it as a team policy.

Squash-merge workflows make this choice less important: the PR's inner history is thrown away at merge time, so a merge from main into your branch leaves no trace in main.

Terminal

```
# Private branch: rebase onto the latest main
git fetch origin
git rebase origin/main
git push --force-with-lease                  # needed because history was rewritten (Q1065)

# Shared branch or reviewed PR: merge, no rewriting
git fetch origin
git merge origin/main
git push

# Only allow fast-forwards on shared branches so surprises fail loudly
git config --global pull.ff only
```


© TechTribers. For personal use only. Do not redistribute.

Section 03

Undoing Things and Recovering Work

The undo cheat sheet with reset modes, amending, recovering with the reflog, committing to the wrong branch, restoring deleted branches and files, bisect, finding who changed what, purging secrets from history and repository health.

Questions 1055–1063 · 9 questions



⏪ **Q1055** GT-021 • Intermediate

reset vs revert vs restore vs clean: which undo do you use when?

⚡ **Quick answer**

`git revert <sha>` adds a **new commit** that undoes an old one: safe for shared history. `git reset` **moves the branch pointer**: `--soft` keeps changes staged, `--mixed` (default) keeps them unstaged, `--hard` also **discards working-tree changes**: for local, unpublished commits only. `git restore` replaces **files**, not history. `git clean` deletes **untracked** files.

Terminal

```
# Undo a commit that is already pushed: add a counter-commit
git revert 9f3a7c1

# Undo local commits (not pushed)
git reset --soft  HEAD~1      # uncommit; changes stay STAGED
git reset         HEAD~1      # uncommit; changes stay in the working tree (mixed)
git reset --hard  HEAD~1      # uncommit AND throw the changes away

# Undo file changes
git restore src/Order.java            # discard unstaged edits to a file
git restore --staged src/Order.java   # unstage

# Remove untracked files (always dry-run first)
git clean -nd                          # show what would be deleted
git clean -fd                          # delete untracked files and directories
```


© TechTribers. For personal use only. Do not redistribute.

git reset <commit>: what moves

	branch / HEAD	index	working tree
`--soft`	moves	unchanged (staged)	unchanged
`--mixed (default)`	moves	reset to commit	unchanged
`--hard`	moves	reset to commit	OVERWRITTEN: edits lost

revert adds a NEW commit that undoes an old one (safe for pushed history).
restore changes FILES, not history · clean deletes UNTRACKED files (run git clean -nd first).

⚠️ **Interview trap**

`git reset --hard` destroys **uncommitted** work for good: the reflog only tracks commits. Committed work can be recovered (Q1057); an uncommitted edit that was never staged cannot.

✏️ **Q1056** GT-022 • Basic

How do you amend the last commit, fix an older one, or change author and message?

⚡ **Quick answer**

`git commit --amend` replaces the last commit with a new one containing the staged changes and/or a new message (new hash). For an **older** commit use a **fixup commit plus autosquash** or `rebase -i` with `reword` / `edit`. Amend and rebase are fine for **unpublished** commits; after pushing, a force-push is required (Q1065).

Terminal

```
git commit --amend -m "Fix refund rounding"       # new message
git add src/RefundService.java
git commit --amend --no-edit                        # add a forgotten file, keep the message
git commit --amend --reset-author                   # use the current configured identity

git commit --fixup 3b9c1d2                          # fix an OLDER commit
git rebase -i --autosquash origin/main

# Change the author of the last 3 commits
git rebase -r HEAD~3 --exec 'git commit --amend --no-edit --reset-author'

git push --force-with-lease                         # only if these commits were already pushed
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Commit hooks run again on `--amend`. If a hook modifies files (a formatter), add them and amend once more before pushing.

Q1057 GT-023 • Intermediate

How does the reflog work, and how do you recover commits after `reset --hard` or a bad rebase?

> ⚡ **Quick answer**
> The **reflog** is a **local** journal of where `HEAD` and each branch pointed over time, updated on commit, checkout, reset, rebase and merge. Commits that are no longer reachable from any branch are **still in the repository** and findable through it: by default, entries expire after **90 days** (reachable) and **30 days** (unreachable). Find the old tip with `git reflog`, then point a branch or `reset --hard` at it.

The reflog is per clone and is not pushed. It can't recover **uncommitted** changes, and it can't help on another machine. A freshly cloned repository has no useful reflog at all.

Terminal

```
git reflog                                  # HEAD's recent positions
# 7d3f2a1 HEAD@{0}: reset: moving to HEAD~3
# a91c4e8 HEAD@{1}: commit: Add refund validation     <- this is what I lost
git branch rescue a91c4e8                    # keep it under a name
git reset --hard HEAD@{1}                    # or move the branch back

git reflog show feature/refunds              # a specific branch's history
git reset --hard ORIG_HEAD                   # undo a rebase or merge just finished

git config --global gc.reflogExpire 180.days
git config --global gc.reflogExpireUnreachable 90.days
```

Q1058 GT-024 • Basic

You committed to the wrong branch (for example main instead of a feature branch). How do you fix it?

> ⚡ **Quick answer**
> If you **haven't pushed**: first create a branch at the current commit so nothing can be lost, then move the original branch back. If you **have pushed** to a shared branch, don't rewrite it: **revert** on that branch and **cherry-pick** the commit onto the right branch.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
# Not pushed: the commits are on main, they belong on feature/refunds
git branch feature/refunds            # 1. a new branch right here keeps the commits safe
git reset --hard origin/main          # 2. move main back to where the remote is
git switch feature/refunds            # 3. continue working

# Only one commit, branch already exists
git switch feature/refunds
git cherry-pick main                   # copy the commit at main's tip
git switch main && git reset --hard HEAD~1

# Already pushed to main
git revert 9f3a7c1 && git push        # undo on main (shared history stays intact)
git switch feature/refunds && git cherry-pick 9f3a7c1
```

💡 **Pro tip**

Step 1 matters most: creating the branch **before** resetting turns a risky recovery into a simple move.

🗑️ **Q1059** GT-025 • Intermediate

How do you recover a deleted branch, a deleted file or a dropped stash?

⚡ **Quick answer**

Deleted **branch**: the commits still exist; find the tip in the reflog (or the hash printed by `git branch -D`) and recreate the branch. Deleted **file**: find the commit that removed it with `git log --diff-filter=D` and restore it from that commit's parent. Dropped **stash**: it's an unreachable commit; list dangling commits with `git fsck` and apply the right one.

Terminal

```
# Deleted branch
git branch -D feature/refunds        # Deleted branch feature/refunds (was 7d3f2a1).
git branch feature/refunds 7d3f2a1   # recreate it from the printed hash
git reflog | grep "feature/refunds"  # if you did not note the hash

# Deleted file
git log --diff-filter=D --oneline -- src/main/java/LegacyTax.java
git restore --source=4c5d6e7^ -- src/main/java/LegacyTax.java   # the commit BEFORE the deletion

# Dropped stash
git fsck --no-reflog | awk '/dangling commit/ {print $3}'       # candidates
git show 9b8a7c6                     # inspect, then
git stash apply 9b8a7c6
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Running `git gc --prune=now` or `git reflog expire --expire=now --all` before checking. Those make dangling commits truly unrecoverable.

🎯 Q1060 GT-026 • Intermediate

How does `git bisect` find the commit that introduced a bug, and how do you automate it?

⚡ **Quick answer**

`git bisect` does a **binary search** over history: you mark one **bad** and one **good** commit, Git checks out the midpoint, you test and mark it, and the range halves each time; 1,000 commits need about 10 steps. `git bisect run <script>` automates it: exit code **0 = good**, **1–127 except 125 = bad**, **125 = skip** (cannot be tested). Always `git bisect reset` afterwards.

Terminal

```
git bisect start
git bisect bad                        # current commit is broken
git bisect good v2.3.0                # last known good release
# Git checks out a midpoint: test, then
git bisect good                       # or: git bisect bad / git bisect skip

# Fully automatic with a failing test
git bisect start HEAD v2.3.0
git bisect run mvn -q -Dtest=RefundServiceTest test
git bisect log                        # the steps taken
git bisect reset                      # return to where you started

# A script that treats non-compiling commits as "skip" rather than "bad"
cat > bisect-test.sh <<'EOF'
#!/bin/sh
mvn -q -DskipTests compile || exit 125
mvn -q -Dtest=RefundServiceTest test
EOF
```

💡 **Pro tip**

Small, atomic commits and a clean `--first-parent` history make bisect dramatically more useful: the culprit commit is then a small, understandable change.

© TechTribers. For personal use only. Do not redistribute.

Q1061 GT-027

• Intermediate

How do you find who changed something, when, and why?

⚡ Quick answer

Combine tools: `git blame` for the last change to each line (add `-w -C` to skip whitespace and detect moved lines); `git log -S` / `-G` to find when a string or pattern entered or left the code; `git log -L` for the history of a function; `git show <sha>` for the full change and message; and the pull request or ticket linked in the message for the **why**. Blame answers "who touched it last", which is often a formatting commit, not the author of the logic.

Commit messages that explain **why** (Q1069) and link the ticket are what make this archaeology possible. `git log` is only as useful as what people wrote into it.

Terminal

```
git blame -w -C -C -L 120,160 src/main/java/PricingService.java
git log -S"MAX_RETRIES" --oneline -- src/main/java      # added/removed the symbol here
git log -L :applyDiscount:src/main/java/PricingService.java --oneline
git show 5e6f7a8 --stat                                  # what else did that commit change?
git log --oneline 5e6f7a8^..5e6f7a8                      # just that commit

# Skip bulk-formatting commits in blame
echo "a1b2c3d4e5f6  # apply google-java-format" >> .git-blame-ignore-revs
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

Q1062 GT-028

• Advanced

How do you remove a secret or a large file from Git history?

⚡ Quick answer

First **rotate the secret**: assume it's compromised, because history is copied to every clone, fork and cache. Then rewrite history with `git filter-repo` (install separately; `filter-branch` is discouraged) to delete the path or replace the text, force-push all refs, and have everyone **re-clone**. Ask the hosting provider to purge cached views and pull-request refs. Prevent the next one with pre-commit secret scanning and the host's push protection.

Terminal

```
# 0. Rotate the credential NOW. Everything below is clean-up, not a fix.

# Find the big blobs
git rev-list --objects --all \
  | git cat-file --batch-check='%(objecttype) %(objectname) %(objectsize) %(rest)' \
  | awk '$1=="blob" && $3 > 5000000' | sort -k3 -n -r | head

# Rewrite history (work on a fresh clone)
git clone --mirror git@github.com:acme/orders.git && cd orders.git
```


© TechTribers. For personal use only. Do not redistribute.

```
git filter-repo --invert-paths --path config/prod-secrets.yml     # delete a file everywhere
git filter-repo --replace-text expressions.txt     # redact: literal==>***REMOVED***

git push --force --mirror origin    # then handle protected branches, forks, open PRs

# Prevention
gitleaks detect --source .              # also as a pre-commit hook and in CI
```

⚠️ **Interview trap**

Deleting the file in a new commit and calling it done. The secret remains in every earlier commit, and `git log -p` or a clone of any old revision exposes it. Only rotation makes it safe.

Q1063 GT-029 • Advanced

What do `git fsck`, `git gc` and `git prune` do, and what if the repository is corrupted?

⚡ **Quick answer**

`git fsck` verifies the object database and lists **dangling** (unreachable) objects and **missing or corrupt** ones. `git gc` compresses loose objects into packfiles and removes unreachable objects older than the expiry; `git prune` deletes unreachable loose objects. For corruption, the usual fix is simple: because every clone is complete, **fetch the missing objects from the remote** or re-clone and copy over your uncommitted work.

Terminal

```
git fsck --full                     # report problems and dangling objects
git count-objects -vH                # loose vs packed size, garbage
git gc                               # routine: repack, prune old unreachable objects

# Make the repository actually shrink after a history rewrite
git reflog expire --expire=now --all
git gc --prune=now --aggressive      # slow; only after filter-repo / BFG

# Corrupt or missing object
git fsck                             # "missing blob 3f4a..." / "bad object"
git fetch origin                     # often restores what is missing
rm .git/objects/ab/cdef...           # remove a truly corrupt loose object, then fetch again

# Last resort: new clone, then copy changed files across
git clone git@github.com:acme/orders.git orders-fresh
```

⚠️ **Interview trap**

Running aggressive prune/gc as a routine "cleanup". It permanently removes the safety net the reflog gives you, and Git's automatic gc is already enough.

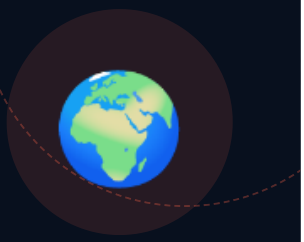
© TechTribers. For personal use only. Do not redistribute.

Section 04

Remotes, Tags and Workflows

Remotes and tracking branches, force-pushing safely, annotated tags and releases, Gitflow versus GitHub flow versus trunk-based development, forks, commit message conventions, submodules, line endings and Git LFS, and what to do when branches diverge.

Questions 1064–1073 · 10 questions



Q1064 GT-030

• Basic

What are remotes, remote-tracking branches and upstream branches?

⚡ Quick answer

A **remote** is a named URL (`origin` by default). A **remote-tracking branch** (`origin/main`) is a **local, read-only** reference to where that branch was at the last fetch: you never commit to it. A local branch's **upstream** (set with `git push -u` or `--set-upstream-to`) is the remote branch it pulls from and pushes to, and is what makes `git status` say "ahead 2, behind 1".

"Branch exists on the remote but I can't see it" nearly always means you haven't fetched: remote-tracking branches only change when you fetch.

Terminal

```
git remote -v                                  # names and URLs
git remote add upstream https://github.com/acme/orders.git
git remote set-url origin git@github.com:me/orders.git
git remote rename origin acme

git push -u origin feature/refunds             # create + set upstream
git branch -vv                                 # each branch, its upstream, ahead/behind
git branch --set-upstream-to=origin/main main

git fetch --prune                              # remove refs of branches deleted on the remote
git ls-remote origin                           # what does the remote have?
git config --global push.autoSetupRemote true  # first push sets the upstream itself
```


© TechTribers. For personal use only. Do not redistribute.

Q1065 GT-031

• Advanced

When is force-pushing acceptable, and what do `--force-with-lease` and `--force-if-includes` add?

⚡ Quick answer

A plain `git push --force` **overwrites** the remote branch with yours, discarding any commits others pushed. It's acceptable only on **your own branch** after rewriting history (rebase, amend, squash). `--force-with-lease` refuses if the remote branch **moved since you last fetched it**. Caveat: a background `git fetch` (an IDE does this) updates your remote-tracking ref, so the lease passes even though you never saw the new commits. `--force-if-includes` (Git 2.30) additionally requires that the remote tip is **in your local history**.

Terminal

```
git push --force                                  # never: overwrites whatever is there
git push --force-with-lease                        # safer: fails if the remote moved
git push --force-with-lease --force-if-includes    # safest convenient form
git push --force-with-lease=feature/refunds:7d3f2a1 # explicit: expect exactly this commit

# Make the safe form the habit
git config --global alias.pushf "push --force-with-lease --force-if-includes"

# Protect shared branches on the server (branch protection / rulesets):
#   block force pushes and deletions on main, release/*
```

⚠️ Interview trap

Force-pushing a shared branch after a rebase "because Git told me to". The push is rejected because the histories diverged; force is only right if you're certain the other side's commits are ones you've deliberately replaced.

Q1066 GT-032

• Intermediate

What is the difference between lightweight and annotated tags, and how do you use tags for releases?

⚡ Quick answer

A **lightweight tag** is just a ref to a commit. An **annotated tag** is a full object with tagger, date and message, and can be **signed**. Use annotated tags for releases (`git describe` and release tooling prefer them). Tags aren't pushed by default: push one by name, or use `--follow-tags` to push annotated tags that point at commits you're pushing. Tags are **meant to be immutable**: retagging a published version breaks everyone who already fetched it.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
git tag v2.4.0                                  # lightweight
git tag -a v2.4.0 -m "Release 2.4.0"            # annotated (use this for releases)
git tag -s v2.4.0 -m "Release 2.4.0"            # signed (Q1075)
git tag -a v2.3.5 4c5d6e7 -m "Backfill tag"     # tag an older commit

git push origin v2.4.0                          # push one tag
git push --follow-tags                        # commits + annotated tags reachable from them
git tag -l "v2.*" --sort=-v:refname             # newest version first
git describe --tags                             # v2.4.0-12-g9f3a7c1: 12 commits after v2.4.0

git tag -d v2.4.0                               # delete locally
git push origin --delete v2.4.0                 # delete on the remote (avoid for releases)
```

💡 **Pro tip**

`git push --tags` pushes **every** local tag, including experiments you never meant to publish. Prefer pushing the specific tag.

Q1067 GT-033

• Intermediate

Gitflow vs GitHub flow vs trunk-based development: which should a team use?

⚡ **Quick answer**

Gitflow: long-lived `main` and `develop`, plus `feature`, `release` and `hotfix` branches; fits **scheduled, versioned releases** and several supported versions, at the cost of heavy merging. **GitHub flow**: `main` is always deployable; short-lived branches go through a pull request and deploy after merge; fits continuous delivery. **Trunk-based development**: everyone integrates into `main` at least daily through tiny branches or directly, hiding unfinished work behind **feature flags**; it needs strong CI and is associated with the best delivery performance in DORA research.

Rule of thumb: **web services and microservices deploying continuously → trunk-based or GitHub flow.** Libraries, mobile apps or products with multiple supported versions → release branches (a light Gitflow). The worst outcome is long-lived feature branches in any workflow: integration pain grows with branch age.

Terminal

```
# GitHub flow / trunk-based in practice
git switch -c feature/refund-endpoint main
# small commits, merged within a day or two
git push -u origin feature/refund-endpoint        # open PR, CI runs, review, merge
# main deploys automatically; unfinished features sit behind a flag:
#   if (flags.isEnabled("refunds-v2")) { ... }

# Gitflow ceremony for comparison
git switch -c release/2.4 develop
git switch main && git merge --no-ff release/2.4 && git tag -a v2.4.0 -m "2.4.0"
git switch develop && git merge --no-ff release/2.4   # merge back, again
```


© TechTribers. For personal use only. Do not redistribute.

Branch lifetime and release model

Trunk-based	main + branches under 1–2 days, feature flags daily integration · needs strong CI
GitHub flow	main always deployable, short PR branches deploy after merge · simple
Gitflow	main + develop + feature / release / hotfix scheduled, versioned releases · heavy merging

Continuous delivery / microservices → trunk-based or GitHub flow. Several supported versions → release branches.
Worst case in any model: long-lived feature branches. Integration pain grows with branch age.

Q1068 GT-034 • Basic

How does the fork-and-pull-request workflow work, and how do you keep a fork in sync?

⚡ **Quick answer**

You **fork** the upstream repository on the hosting site, **clone your fork** (`origin`), add the original as `upstream` , work on a **feature branch** (never on your fork's `main`), push it to `origin` and open a pull request against upstream. To sync, fetch `upstream` and fast-forward your `main` , then rebase or merge it into your feature branch.

Terminal

```
git clone git@github.com:me/orders.git && cd orders
git remote add upstream https://github.com/acme/orders.git

git switch -c fix/rounding
# ... commits ...
git push -u origin fix/rounding                  # then open the PR to acme/orders

# Later: bring upstream changes into your fork
git fetch upstream
git switch main
git merge --ff-only upstream/main                # fails if you committed on main by mistake
git push origin main
git switch fix/rounding && git rebase main

gh repo fork acme/orders --clone                 # GitHub CLI does the first steps for you
```

💡 **Pro tip**

Keeping your fork's `main` identical to upstream (fast-forward only) means a failed `--ff-only` instantly tells you that you committed to main by mistake.

© TechTribers. For personal use only. Do not redistribute.

Q1069 GT-035

• Basic

What makes a good commit and commit message, and what are Conventional Commits?

⚡ Quick answer

A good commit is **atomic** (one logical change that builds and passes tests) with a message that explains **why**. Convention: a **subject** of 50–72 characters in the imperative ("Add refund endpoint"), a blank line, then a body wrapped at about 72 characters explaining motivation and trade-offs, then trailers (`Closes #482` , `Co-authored-by:`). **Conventional Commits** adds a machine-readable prefix (`feat:` , `fix:` , `docs:` , `refactor:` , `feat!:` for breaking changes) that tools use to calculate versions and changelogs.

Tools that consume the convention: `semantic-release` , `release-please` and `git-cliff` compute the next semantic version (`fix` → patch, `feat` → minor, `!` → major) and write the changelog from the history.

Terminal

```
feat(refunds): add partial refund endpoint

Customers could only refund a whole order, which forced support to issue
manual credit notes. Allow refunding any line item up to the paid amount.
Amounts are validated against already-refunded totals in one transaction
to prevent double refunds under concurrent requests.

Closes #482
Co-authored-by: Ravi Sharma <ravi@example.com>

# Breaking change
feat(api)!: remove the deprecated /v1/orders/search endpoint

BREAKING CHANGE: clients must use /v2/orders?query=...

git config --global commit.template ~/.gitmessage          # a reminder template
# commit-msg hook + commitlint reject messages that do not follow the format
```

Q1070 GT-036

• Advanced

Submodules vs subtrees vs publishing a library: how do you share code between repositories?

⚡ Quick answer

A **submodule** pins another repository at an **exact commit** inside yours (`.gitmodules` plus a special tree entry): precise, but clones, CI and contributors must handle the extra step, and it's easy to push a parent that references a submodule commit nobody pushed. A **subtree copies** the other repo's content (optionally with history) into a directory, so consumers see ordinary files. For Java code, the best answer is usually neither: **publish the library as a versioned artifact** (Maven repository) and depend on it normally.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
# Submodule
git submodule add https://github.com/acme/shared-protos.git proto
git clone --recurse-submodules git@github.com:acme/orders.git     # or after cloning:
git submodule update --init --recursive
git submodule update --remote proto                 # move to the submodule's latest main
git commit -am "Bump shared-protos"                  # the parent records the new commit

# Subtree
git subtree add  --prefix=libs/shared https://github.com/acme/shared.git main --squash
git subtree pull --prefix=libs/shared https://github.com/acme/shared.git main --squash

# Usually better for Java: a versioned dependency
#   <dependency><groupId>com.acme</groupId><artifactId>shared-protos</artifactId>
#   <version>3.2.0</version></dependency>
```

⚠️ **Interview trap**

Pushing the parent repository before the submodule commit it points to. Everyone else's `submodule update` then fails with "not our ref". Push the submodule first, or set `push.recurseSubmodules=check`.

↩️ **Q1071** GT-037 • Intermediate

How do line endings and `.gitattributes` work, and how do you avoid CRLF/LF diffs?

⚡ **Quick answer**

Windows uses CRLF and Unix LF, so the same file can appear changed on every line. Put the policy **in the repository** with `.gitattributes` (`* text=auto eol=lf` , `*.bat text eol=crlf` , binaries marked `binary`) so every developer and CI agent behaves the same; `core.autocrlf` is a per-machine setting and easy to get wrong. `git add --renormalize .` re-applies the rules to existing files. Shell scripts with CRLF fail on Linux (`bad interpreter`), and the executable bit on `mvnw` must be set in Git.

Terminal

```
# .gitattributes
* text=auto eol=lf
*.bat   text eol=crlf
*.cmd   text eol=crlf
*.jar   binary
*.png   binary

git add --renormalize .                             # apply the rules to existing files
git commit -m "Normalize line endings"
git ls-files --eol | head                           # i/lf w/lf = index and working tree both LF

git update-index --chmod=+x mvnw gradlew            # keep wrappers executable
git config --global core.autocrlf input     # fallback: CRLF to LF on commit (mac/Linux)
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Add the normalisation commit to `.git-blame-ignore-revs` (Q1061) so blame still points at the real authors.

📦 **Q1072** GT-038 • Intermediate

What is Git LFS, how does it work, and what are its pitfalls?

> ⚡ **Quick answer**
> **Git LFS** stores large files (design assets, datasets, binaries) **outside** the repository: Git tracks a tiny **pointer file** and the real content lives on an LFS server, downloaded at checkout. `git lfs track "*.psd"` records the rule in `.gitattributes`. It keeps clones fast, but storage and bandwidth are usually **quota-limited and billed**, history that already contains big files needs `git lfs migrate`, and CI must have LFS enabled or it checks out pointer text instead of files.

Terminal

```
git lfs install                          # once per machine: registers the filters
git lfs track "*.psd" "datasets/*.parquet"
git add .gitattributes                    # the tracking rules are committed
git add design/logo.psd && git commit -m "Add logo source"
git lfs ls-files                          # which files are stored in LFS

# Move files that were committed normally into LFS (rewrites history!)
git lfs migrate import --include="*.psd" --everything

# CI: GitHub Actions
#   - uses: actions/checkout@v4
#     with: { lfs: true }
```

> ⚠️ **Interview trap**
> Putting build output (`target/*.jar`) or dependencies in LFS. Build artifacts don't belong in Git at all: publish them to an artifact repository (Q1085).

🧭 **Q1073** GT-039 • Intermediate

Your local branch and the remote branch have diverged. What do you do?

> ⚡ **Quick answer**
> `git status` says "Your branch and 'origin/feature' have diverged, and have 3 and 2 different commits each": **both sides have commits the other lacks**. First **look** with `git log --left-right`, and make a **backup branch**. Then choose: **rebase** your commits on top of theirs (`git pull --rebase`), **merge**, or, if you know your local history is wrong (for example after an accidental rewrite), **reset to the remote** (`git reset --hard origin/feature`).

© TechTribers. For personal use only. Do not redistribute.

Diverging after **you** rebased or amended and pushed is the classic cause: the remote still has the old commits. In that case you want `git push --force-with-lease` (Q1065), not a pull.

Terminal

```
git fetch origin
git status -sb                          # ## feature...origin/feature [ahead 3, behind 2]
git log --oneline --left-right --graph feature...origin/feature
git branch backup/feature-before-fix                # safety net

# Usual fix: put my commits on top of theirs
git pull --rebase origin feature

# They pushed commits I want, I made commits they must receive: merge
git merge origin/feature

# My local branch is wrong; discard it and take the remote's
git reset --hard origin/feature
```

Section 05

Hooks, Security and Scale

Client and server hooks, signed commits, packfiles and why repositories get large, shallow, partial and sparse clones for monorepos, SHA-1 versus SHA-256, and credentials with multiple accounts.

Questions 1074–1079 · 6 questions



Q1074 GT-040

• Intermediate

What are Git hooks, and how do you share them across a team?

⚡ Quick answer

Hooks are scripts Git runs at points in its workflow. **Client-side**: `pre-commit` (lint, format, secret scan), `commit-msg` (validate the message), `pre-push` (run tests). **Server-side**: `pre-receive` / `update` / `post-receive` (policy checks and notifications; hosted platforms expose these as branch protection and CI). Hooks in `.git/hooks` are **not versioned or cloned**, so share them with `core.hooksPath` pointing at a committed directory, or a framework (pre-commit, Husky, Lefthook).

Terminal

```
mkdir .githooks
cat > .githooks/pre-commit <<'EOF'
#!/bin/sh
# Reject commits that would fail the formatting check in CI
```


© TechTribers. For personal use only. Do not redistribute.

```
mvn -q spotless:check || { echo "Run: mvn spotless:apply"; exit 1; }
EOF
chmod +x .githooks/pre-commit
git config core.hooksPath .githooks      # each developer runs this once (or via a setup script)

git commit --no-verify -m "WIP"          # hooks are skippable: enforce the same checks in CI

# pre-commit framework: config is committed, install once
pre-commit install
pre-commit run --all-files
```

⚠️ **Interview trap**

Treating hooks as enforcement. Anyone can bypass them with `--no-verify`, and server-side hooks may not exist on your hosting platform. Use hooks for fast feedback and **CI plus branch protection** for the rules that must hold.

✍️ **Q1075** GT-041 • Intermediate

Why and how do you sign commits (GPG, SSH, gitsign), and how do you verify them?

⚡ **Quick answer**

`user.name` and `user.email` can be set to anything, so **authorship is trivially forgeable**. A **signature** proves that a specific key created the commit. Options: **GPG** (the classic), **SSH signing** (Git 2.34: reuse your SSH key, much simpler), and **Sigstore gitsign** (keyless, identity from OIDC). Hosts show a "Verified" badge when the public key is registered, and branch protection can require signed commits.

Signing proves **who**, not that the code is good. It matters most for release tags and commits to protected branches, and for supply-chain requirements (SLSA, internal compliance). Squash merges made through the web UI are signed by the platform, not by the author.

Terminal

```
# SSH signing: simplest setup
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true            # sign every commit
git config --global tag.gpgsign true

git commit -S -m "Rotate payment gateway key"      # sign explicitly
git log --show-signature -1
git verify-commit HEAD

# Verification locally needs the trusted keys listed
echo "asha@example.com $(cat ~/.ssh/id_ed25519.pub)" >> ~/.ssh/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
```


© TechTribers. For personal use only. Do not redistribute.

Q1076 GT-042

• Advanced

How are objects stored in packfiles, and why do repositories get huge?

⚡ Quick answer

New objects start as **loose** zlib-compressed files. `git gc` (run automatically) bundles them into **packfiles** and stores similar objects, such as successive versions of a file, as **deltas** against each other, so storage is far smaller than the snapshot model suggests. Repositories get huge because **everything ever committed stays in history**: one 200 MB binary added and later deleted still costs 200 MB in every clone. Fix with history rewriting (`filter-repo`), LFS, or partial clones.

Terminal

```
git count-objects -vH                          # size-pack: the packed history
du -sh .git                                     # the real on-disk footprint

# Largest objects anywhere in history
git rev-list --objects --all \
  | git cat-file --batch-check='%(objecttype) %(objectname) %(objectsize) %(rest)' \
  | sort -k3 -n -r | head -10

git repack -a -d --depth=250 --window=250      # slower, tighter pack
git gc --auto                                   # what Git does by itself after many loose objects

# Typical culprits: committed jars, database dumps, node_modules, IDE caches, videos
```

💡 Pro tip

Check repository size in CI (a job that fails when `.git` exceeds a budget). Preventing a big file from entering is far cheaper than purging it later.

Q1077 GT-043

• Advanced

How do shallow, partial and sparse clones help with large repositories and monorepos?

⚡ Quick answer

Shallow clone (`--depth 1`): only recent history; fast for CI, but breaks `git describe` , `blame` and merge-base calculations. **Partial clone** (`--filter=blob:none`): **full history, file contents fetched on demand**; the usual choice for developers. **Sparse checkout** (`git sparse-checkout set <dirs>`): only chosen directories appear in the working tree. `git maintenance` , **fsmonitor** and **Scalar** (`scalar clone`) bundle these optimisations for very large monorepos.

Terminal

```
# CI: latest commit only
git clone --depth 1 https://github.com/acme/platform.git
```


© TechTribers. For personal use only. Do not redistribute.

```
git fetch --unshallow                                  # later, if tags or history are needed

# Developers: all history, blobs on demand, only one service checked out
git clone --filter=blob:none --sparse https://github.com/acme/platform.git
cd platform
git sparse-checkout set --cone services/orders libs/common

# Keep a big repository fast in the background
git maintenance start                                  # commit-graph, prefetch, incremental repack
git config core.fsmonitor true                         # watch file changes instead of scanning
scalar clone https://github.com/acme/platform.git      # all of the above, preconfigured
```

⚠️ **Interview trap**

Using `--depth 1` in a pipeline that computes versions from `git describe` or diffs against `main`. The tags and the base commit aren't there, so versions come out as a bare hash and change detection silently covers everything (Q1081).

#️⃣ Q1078 GT-044 • Advanced

Does Git's use of SHA-1 make it insecure, and what is SHA-256 mode?

⚡ **Quick answer**

Git identifies objects by hash, and SHA-1 has been broken for collisions (the SHAttered attack, 2017). Git's response was **sha1dc**, hardened SHA-1 that **detects collision-attack patterns** and refuses them, so the practical risk is low. Separately, Git supports **SHA-256 repositories** (`git init --object-format=sha256`), which are **not interoperable** with SHA-1 repositories, and hosting-platform support is still limited. Integrity comes from the hash chain; **authenticity** comes from signatures (Q1075).

Interview framing: hashing in Git is for **identity and integrity**, not for secrecy or authenticity. Anyone can create a commit with any author, which is why signatures and protected branches exist.

Terminal

```
git --version
git rev-parse HEAD                                     # 40 hex characters = SHA-1
git rev-parse --show-object-format                     # sha1 | sha256

git init --object-format=sha256 secure-repo            # experimental: 64 hex characters
cd secure-repo && git commit --allow-empty -m "init"
git rev-parse HEAD                                     # 64 hex characters
```


© TechTribers. For personal use only. Do not redistribute.

🔑 **Q1079** GT-045 • Intermediate

SSH keys vs HTTPS tokens vs credential helpers: how do you authenticate, including multiple accounts?

⚡ **Quick answer**

SSH: a key pair, public key registered on the host, private key protected by a passphrase and `ssh-agent`; no tokens in URLs. **HTTPS**: a **personal access token** or OAuth login stored by a **credential helper** (Git Credential Manager, macOS keychain), never typed into the URL. For **multiple accounts** (personal and work), use SSH host aliases or `includeIf` per directory so each folder picks the right identity and key.

Terminal

```
ssh-keygen -t ed25519 -C "asha@example.com"            # modern default key type
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
ssh -T git@github.com                                    # test the connection

# ~/.ssh/config: two accounts on one host
Host github-work
    HostName github.com
    IdentityFile ~/.ssh/id_work
Host github-personal
    HostName github.com
    IdentityFile ~/.ssh/id_personal

git clone git@github-work:acme/orders.git                # uses the work key

# ~/.gitconfig: identity by directory
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work                              # user.email = asha@acme.com

git config --global credential.helper manager             # HTTPS: store tokens safely
```

⚠️ **Interview trap**

Embedding a token in the remote URL (`https://TOKEN@github.com/...`). It's stored in plain text in `.git/config`, shows up in logs and `git remote -v`, and ends up in shared screenshots.

© TechTribers. For personal use only. Do not redistribute.

Section 06

Git in Teams, CI/CD and Java Projects

Branch protection and CODEOWNERS, Git in CI pipelines, release and hotfix branches, embedding commit info in Spring Boot builds, a correct .gitignore for Java projects and tag-driven Maven versioning.

Questions 1080–1086 · 7 questions



Q1080 GT-046

• Intermediate

What do branch protection, rulesets and CODEOWNERS enforce?

⚡ **Quick answer**

Branch protection turns team conventions into **rules the server enforces** on `main` and `release/*`: required **pull requests and approvals**, **status checks** that must pass, branches that must be **up to date**, **linear history**, **signed commits**, and **no force pushes or deletions**. A `CODEOWNERS` file maps paths to people or teams, who are then **automatically requested** for review (and can be required to approve).

Terminal

```
# .github/CODEOWNERS (last matching rule wins)
*                       @acme/backend
/services/payments/     @acme/payments-team
/db/migration/          @acme/dba @asha
*.gradle                @acme/build-tools
/docs/                  @acme/tech-writers

# Typical rules on main
#   require a pull request, 1-2 approvals, dismiss stale approvals on new commits
#   require status checks: build, unit-tests, sonar-quality-gate
#   require branches to be up to date (or use a merge queue)
#   require linear history; block force pushes and branch deletion
#   include administrators: rules that admins can bypass are only suggestions
```

💡 **Pro tip**

A **merge queue** tests each PR on top of the current main plus the PRs ahead of it, which prevents two individually-green PRs from breaking main together.

© TechTribers. For personal use only. Do not redistribute.

Q1081 GT-047

• Advanced

How do you use Git correctly in CI pipelines (fetch depth, changed files, versions)?

⚡ Quick answer

CI systems clone **shallowly** by default, which breaks anything needing history: set the fetch depth (`fetch-depth: 0` in GitHub Actions) for jobs that compute versions or diffs. Test what will actually be merged (`refs/pull/N/merge`). Detect changed modules with `git diff --name-only origin/main...HEAD` to run only the affected builds in a monorepo. Derive versions from tags with `git describe` , and stop pipelines retriggering themselves with `[skip ci]` .

Prefer **tag-triggered** release pipelines: pushing the annotated tag `v2.5.0` builds, tests, publishes and creates the release notes, so the tag is the single source of truth for what shipped.

Terminal

```
# .github/workflows/build.yml
#   - uses: actions/checkout@v4
#     with:
#       fetch-depth: 0              # full history and tags: versioning, diffs, blame

# In the job: which modules changed in this PR?
git fetch origin main
CHANGED=$(git diff --name-only origin/main...HEAD | cut -d/ -f1-2 | sort -u)
echo "$CHANGED"                     # e.g. services/orders  libs/common

# Version from the nearest tag: v2.4.0-12-g9f3a7c1
VERSION=$(git describe --tags --always --dirty)

# A release commit made by CI must not trigger another run
git commit -m "chore(release): 2.5.0 [skip ci]"
```

Q1082 GT-048

• Intermediate

How do release branches, hotfixes and backports work in practice?

⚡ Quick answer

Cut `release/2.4` from `main` when the version is feature-complete; only **fixes** go on it, each cherry-picked from `main` with `-x` . For a production bug, branch from the **tag running in production**, fix on `main` first (so it can't regress), cherry-pick to the release branch, then **tag** the patch version. Supported old versions get the same treatment on their own `release/*` branch.

Terminal

```
git switch -c release/2.4 main              # stabilise 2.4
git push -u origin release/2.4
```


© TechTribers. For personal use only. Do not redistribute.

```
# Production bug in 2.4.0
git switch -c fix/refund-rounding main
# ... fix, test, PR, merge to main (upstream first) ...
git switch release/2.4
git cherry-pick -x 9f3a7c1                          # backport the fix commit
git tag -a v2.4.1 -m "Release 2.4.1"
git push origin release/2.4 v2.4.1

git branch -r --contains 9f3a7c1                     # which branches already have this fix?
git log --oneline v2.4.0..release/2.4                # what is in the patch release?
```

💡 **Pro tip**

`git cherry` and `git log --cherry-pick --left-right main...release/2.4` list commits present on one branch but missing on the other: a quick check that no fix was forgotten.

🌱 **Q1083** GT-049 • Intermediate

How do you embed Git commit information in a Spring Boot build (git.properties, /actuator/info)?

⚡ **Quick answer**

Generate a `git.properties` file at build time with `git-commit-id-maven-plugin` (or the Gradle plugin). Spring Boot reads it automatically and exposes it through `/actuator/info`, so you can always answer "which commit is running in production?". Add `build-info` from the Spring Boot plugin for version and build time.

Terminal

```
<!-- pom.xml -->
<plugin>
  <groupId>io.github.git-commit-id</groupId>
  <artifactId>git-commit-id-maven-plugin</artifactId>
  <executions><execution><goals><goal>revision</goal></goals></execution></executions>
  <configuration>
    <generateGitPropertiesFile>true</generateGitPropertiesFile>
    <failOnNoGitDirectory>false</failOnNoGitDirectory>
  </configuration>
</plugin>
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions><execution><goals><goal>build-info</goal></goals></execution></executions>
</plugin>

# application.properties
management.endpoints.web.exposure.include=health,info
management.info.git.mode=full              # branch, commit id, message, time, dirty flag
management.info.build.enabled=true
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Building from a shallow or detached CI checkout without tags: the properties are generated, but `git.tags` and `git.closest.tag.name` come out empty. Fetch tags in the build job (Q1081).

☕ **Q1084** GT-050 • Basic

What should a .gitignore and the committed files look like for a Java/Maven/Gradle project?

⚡ **Quick answer**

Ignore build output (`target/` , `build/` , `.gradle/`), compiled classes and logs, IDE and OS files, local configuration and anything secret. **Commit** the build definition (`pom.xml` , `build.gradle` , version catalogs), the **wrapper** (`mvnw` , `.mvn/wrapper/` , `gradlew` , `gradle/wrapper/gradle-wrapper.jar`) so builds don't depend on a local installation, example configuration (`application-local.yml.example`) and migrations.

Terminal

```
# .gitignore
# Build output
target/
build/
out/
.gradle/
*.class
*.log
*.jar
!.mvn/wrapper/maven-wrapper.jar          # the wrapper jars ARE committed
!gradle/wrapper/gradle-wrapper.jar

# IDE and OS
.idea/
*.iml
.vscode/
.classpath
.project
.settings/
.DS_Store

# Local config and secrets
.env
application-local.yml
application-local.properties
*.pem
*.p12
*.jks
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Start from the Java template at gitignore.io or GitHub's `gitignore` repository, then adapt. Keep personal editor clutter in your global ignore (Q1041) rather than every project's file.

Q1085 GT-051 • Advanced

How do you version a Maven project from Git tags (release plugin vs tag-driven CI)?

> ⚡ **Quick answer**
> The **Maven release plugin** edits `pom.xml` to the release version, commits, **tags**, bumps to the next SNAPSHOT and commits again: two commits per release, and it often fails halfway on a CI server. The modern approach is **tag-driven**: the pom uses `${revision}`, the pipeline passes the version taken from the tag (`mvn -Drevision=2.5.0`), and the `flatten-maven-plugin` makes published poms resolvable. The Git tag is the single source of truth and no version-bump commits are needed.

Pair it with Conventional Commits and `release-please` or `semantic-release` and even the **next version number** is computed for you from the merged PRs, with the tag created by the release pull request.

Terminal

```
<!-- pom.xml: version comes from a property -->
<version>${revision}</version>
<properties><revision>0.0.0-SNAPSHOT</revision></properties>

<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>flatten-maven-plugin</artifactId>
  <configuration>
    <updatePomFile>true</updatePomFile>
    <flattenMode>resolveCiFriendliesOnly</flattenMode>
  </configuration>
  <executions><execution><id>flatten</id><phase>process-resources</phase>
    <goals><goal>flatten</goal></goals></execution></executions>
</plugin>

# CI on a tag push v2.5.0
VERSION=${GITHUB_REF_NAME#v}                  # 2.5.0
mvn -B -Drevision=$VERSION -DskipTests=false verify deploy

# Local builds stay 0.0.0-SNAPSHOT, or take the latest tag:
# mvn -Drevision=$(git describe --tags --abbrev=0 | sed s/^v//) verify
```


© TechTribers. For personal use only. Do not redistribute.

Q1086 GT-052

• Advanced

Interview scenario: you accidentally pushed a password to `main`. What do you do, in order?

⚡ **Quick answer**

Treat it as an incident, not a Git problem. **1. Revoke and rotate** the credential immediately. **2. Assess exposure**: audit logs for use of the credential. **3. Remove it from history** (`filter-repo`) only after rotation, force-push, and have everyone re-clone; ask the host to purge cached views and PR refs. **4. Prevent recurrence**: secret scanning (push protection, gitleaks), `.env` handling, a secrets manager. **5. Write it up** blamelessly.

Interviewers use this question to see if you know that **rotation comes first** and that history rewriting is only clean-up. A candidate who starts with `git filter-branch` has the priorities reversed.

Terminal

```
# 1. Rotate in the system that issued it (database, cloud, API) -> old value is dead
# 2. Check whether it was used
#    cloud audit log / database login history since the push time

# 3. Rewrite history on a mirror clone (Q1062), then
git push --force --mirror origin
#    ask collaborators to re-clone; delete stale forks; request cache purge from the host

# 4. Make it hard to repeat
pre-commit install                    # gitleaks / detect-secrets hook
#    enable platform push protection and secret scanning alerts
#    move credentials to a secrets manager; read them from the environment

# What not to do
git rm config/prod.yml && git commit -m "remove password"   # still in history, still exploitable
```


© TechTribers. For personal use only. Do not redistribute.

Series progress

1086 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 22 starts at Q1087.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50
Part 14	Q580–Q679	100
Part 15	Q680–Q739	60
Part 16	Q740–Q799	60
Part 17	Q800–Q849	50
Part 18	Q850–Q924	75
Part 19	Q925–Q974	50
Part 20	Q975–Q1034	60
Part 21	Q1035–Q1086	52

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.

© TechTribers. For personal use only. Do not redistribute.